Logr – Technical Requirements (Server Side)

Version 2.0 · Complete specification for backend design and AI-assisted build · pairs with Logr – Product Requirements (Client Side)

1. Summary and stack

This document says what the server does for every action in Logr, using the same F1 to F13 numbering as the Product Requirements. For each feature it gives the endpoints, validation, database writes, side effects and errors. Requirement IDs (FR-Fx.x) refer to the client document. Either document can be pasted into an AI prompt to generate code or flows.

Layer	Choice
API	Python 3.12, Django 5, Django REST Framework; one project, one Django app per module
Database	PostgreSQL
Cache, throttling, broker	Redis
Background jobs	Celery workers and Celery Beat
Files	S3-compatible storage through django-storages; signed URLs
Auth	djangorestframework-simplejwt; refresh token in an httpOnly secure cookie; blacklist enabled
Passwords	Django Argon2 hasher
Email	Transactional provider, sent from Celery tasks
API docs	drf-spectacular (OpenAPI and Swagger)
Client	Next.js with TanStack Query (consumes this API)

Modules (Django apps): accounts, invitations, cohorts, logbook, extensions, attendance, reviews, supervisors, notifications, adminpanel, audit, uploads.

Golden rules the server enforces

1. There is no daily deadline; the only deadline is the weekly one (Saturday 23:59:59 Africa/Lagos by default), judged on the server clock.

2. A day with no saved content is "not filled yet", never "missed". Only a week can be missed.
3. A week can be submitted only when every working day of that week is filled.
4. Future days cannot be opened or written.
5. Submitted weeks are read-only unless the coordinator returns them or an approved extension reopens them.
6. SWEP attendance only through a coordinator-generated code valid for at most 3 minutes; no timer on the student side.
7. SIWES supervisor reviews, sent in batches whenever the student chooses, through a token link, with no supervisor account.

2. Global conventions

- **Base path** `/api/v1`. JSON only. Pagination on every list (`limit`, `cursor`).
- **Time**: `TIME_ZONE = Africa/Lagos`, `USE_TZ = True`; stored in UTC. Responses include `server_time` so clients display countdowns correctly.
- **Auth**: access token 15 minutes; refresh 7 days with rotation and blacklist. Role comes from the user record only.
- **Permissions**: DRF classes `IsStudent`, `IsCoordinator`, `IsAdmin`, `SupervisorTokenAuthentication`, plus object-level checks (student: own rows; coordinator: students of own department and cohort; admin: all).
- **Errors**: `{ "code": "week_locked", "message": "...", "details": {} }`; validation errors list fields.
- **Idempotency**: submit, approve, deny, send-to-supervisor and generate-code requests return the current state when repeated, unless noted.
- **Concurrency**: state changes run in a transaction with `select_for_update`.
- **Audit**: admin actions, coordinator decisions (extension, return, override, review void) and supervisor submissions write to `audit_logs`.
- **Throttling**: scoped throttles on login, signup, password reset, invitation accept, attendance submit, supervisor token opens.
- **Sanitising**: all free text has HTML stripped and length limits.

3. Time model and derived states

- A cohort has `start_date`, `end_date`, `working_days` (default Mon to Fri), `holidays` (dates), and settings: `deadline_weekday` (default Saturday), `deadline_time` (default 23:59:59), `max_grace_days` (7), `request_window_days` (14),

`max_extensions_per_week` (1), `correction_days` (2), `min_filled_chars` (20), `attendance_max_minutes` (3), `attendance_max_codes_per_day` (3).

- **Weeks** are Monday to Sunday: `week_no = floor((entry_date - monday_of(start_date)) / 7 days) + 1`. Week 1 may start mid-week.
- **Working days of a week** = week dates within `[start_date, end_date]` whose weekday is in `working_days` and not a holiday.
- `deadline_at(week)` = the deadline weekday and time of that week in Lagos.
- `effective_deadline` = `correction_deadline` if status is RETURNED; else `max(deadline_at, extended_until)`.
- **Filled day**: an entry exists for that date with `activities` of at least `min_filled_chars` characters.
- **Editable** means: week status is OPEN or RETURNED and `now <= effective_deadline`.
- **Overdue** = week status OPEN and `now > effective_deadline` while an extension is still possible. **Missed week** = week status OPEN and `now > effective_deadline` with no extension possible (request window closed, extension denied, extension limit reached, or approved extension expired). Both are derived at request time; no job is needed for correctness.
- A **review batch** is the set of submitted weeks not yet sent to the supervisor. It can be any size, so the programme length does not need to be a multiple of four and the student can send weekly, every two weeks, every four weeks or once at the end.

4. State machines

Weekly submission: `OPEN` to `SUBMITTED` (student, only when editable and all working days filled) to `REVIEWED` (coordinator finishes review). `SUBMITTED` to `RETURNED` (coordinator, comment required; sets `correction_deadline = now + correction_days`), then back to `SUBMITTED` before the correction deadline. A returned week past its correction deadline becomes read-only and can still be rated.

Day entry: no row (not filled yet) to `DRAFT` on first save; `DRAFT` follows the week to `SUBMITTED`, `REVIEWED` or `RETURNED`. Entries never have a missed state.

Extension request: `PENDING` to `APPROVED` or `DENIED` (coordinator); `APPROVED` to `EXPIRED` when `new_deadline` passes with the week still OPEN.

Attendance session: `ACTIVE` (now < `expires_at`, not ended) to `ENDED` (timer expiry, End now, or replaced by a new code). **Attendance record**: `PRESENT`, `EXCUSED` (coordinator override), `ABSENT` (nightly finalisation).

Supervisor review batch: created as `SENT` when the student taps Send to supervisor, then `REVIEWED` when the supervisor submits. `SENT` becomes `EXPIRED` when the token lapses; resend returns it to `SENT` with the same weeks. A week belongs to at most one batch; voiding a batch releases its weeks.

Invitation: `PENDING` to `ACCEPTED`, `EXPIRED` or `REVOKED`. **User**: `ACTIVE`, `DEACTIVATED`, `WITHDRAWN`, `RUSTICATED`.

5. Data model

Table	Key fields
users	id, role (STUDENT, COORDINATOR, ADMIN), email unique, password_hash, full_name, status, status_reason, status_note, created_at
departments	id, name
student_profiles	user_id, matric_no unique (case-insensitive), department_id, programme (SWEP, SIWES), cohort_id (nullable), coordinator_id (nullable)
coordinator_profiles	user_id, department_id
invitations	id, email, department_id, token_hash, status, expires_at, invited_by, accepted_at
password_reset_tokens	id, user_id, token_hash, expires_at, used_at
cohorts	id, programme, session, department_id, coordinator_id, start_date, end_date, working_days (array), settings (JSON, see section 3), status (UPCOMING, ACTIVE, CLOSED)
cohort_holidays	id, cohort_id, date, label
weekly_submissions	id, student_id, cohort_id, week_no, status (OPEN, SUBMITTED, RETURNED, REVIEWED), deadline_at, extended_until, submitted_at, is_late, returned_at, correction_deadline, reviewed_at. Unique (student_id, week_no)
logbook_entries	id, student_id, week_id, entry_date, activities, skills, review_text, status (DRAFT, SUBMITTED, REVIEWED, RETURNED), updated_at. Unique (student_id, entry_date)

Table	Key fields
entry_images	id, entry_id, storage_key, thumb_key, size, mime, created_at
entry_reviews	id, entry_id, coordinator_id, rating 1 to 5, comment, reviewed_at. Unique (entry_id)
extension_requests	id, week_id, student_id, reason, attachment_key, status, requested_at, decided_by, decided_at, grace_days, decision_comment, new_deadline. One PENDING per week (partial unique index)
attendance_sessions	id, cohort_id, date, code_hash, salt, started_at, expires_at, ended_at, created_by, duration_minutes
attendance_records	id, student_id, date, session_id, status (PRESENT, EXCUSED, ABSENT), marked_at, marked_by, reason, ip. Unique (student_id, date)
supervisors	id, student_id, name, email, phone, organisation, updated_at
supervisor_reviews	id, student_id, batch_no, supervisor_id, status, sent_at, reviewed_at, supervisor name/email/phone snapshots, review_text, signature_key, signature_type (DRAWN, TYPED), signed_name, reviewer_ip, reviewer_agent. Unique (student_id, batch_no). Weeks are linked through supervisor_review_weeks (review_id, week_id); week_id is unique, so a week belongs to at most one batch.
supervisor_tokens	id, supervisor_review_id, token_hash unique, expires_at, used_at, revoked_at, created_at, last_opened_at, open_count
notifications	id, user_id, type, payload (JSON), read_at, created_at
audit_logs	id, actor_id, action, target_type, target_id, meta (JSON), created_at

Redis keys: `att:code:{session_id}` (plain code with TTL equal to the session, for the coordinator's display only), `att:tries:{session_id}:{student_id}`, throttle counters.

6. F1 Student signup and programme selection

Endpoint: `POST /auth/student/signup`.

1. Validate with a serializer: full name, matric number, email, department id, password (8+ characters, Django validators), programme in (SWEP, SIWES).
2. Matric number and email unique, case-insensitive; return field errors `matric_taken`, `email_taken`.
3. In one transaction create the user (role STUDENT, Argon2 hash) and student profile.
4. Find the cohort with the same programme and department whose status is UPCOMING or ACTIVE and attach `cohort_id` and `coordinator_id`. If none, leave null; the home endpoint then returns `cohort: null`.
5. Issue access and refresh tokens; return the user and profile (never the hash).
6. Throttle by IP. Programme cannot be changed later except by admin (F12).

7. F2 Login, session, password reset, profile

- `POST /auth/login`: look up by email, constant-time password check, reject non-ACTIVE users with `account_withdrawn`, `account_rusticated` or `account_deactivated`, issue tokens, set the refresh cookie. Generic `invalid_credentials` for wrong email or password. Throttle by IP and email with growing delay.
- `POST /auth/refresh`: rotate refresh token; reject blacklisted or inactive users. `POST /auth/logout` blacklists the refresh token.
- `POST /auth/forgot-password`: always return 200. If the user exists, create a random token (hash stored, 1 hour expiry) and queue the email.
- `POST /auth/reset-password`: validate token (unused, unexpired), set the new password, mark token used, blacklist all refresh tokens of that user.
- `GET /me` returns profile; `PATCH /me/password` requires the current password.
- `GET /me` for SIWES students also returns the saved supervisor details.

8. F3 Admin sign-in

- `POST /auth/admin/login` accepts only users with role ADMIN; otherwise `invalid_credentials`.
- Admin accounts are created only by a management command (`createadmin`). No signup endpoint exists. Admin routes require `IsAdmin` and are throttled.

9. F4 Coordinator invitation

1. `POST /admin/invitations` (admin): validate the email is not an existing user and has no pending invitation for that department.
2. Generate `secrets.token_urlsafe(32)`; store only its SHA-256 hash with the email, department, role COORDINATOR and `expires_at = now + 72 hours`. Queue the email containing the raw token in `/accept-invite?token=...`.
3. `POST /admin/invitations/{id}/resend`: revoke the old token, create a new one, queue the email. `DELETE /admin/invitations/{id}`: status REVOKED.
4. `GET /invitations/validate?token=`: hash, find a PENDING unexpired row, return the locked email and department only.
5. `POST /invitations/accept`: in one transaction create the user (role COORDINATOR) and coordinator profile, mark the invitation ACCEPTED, audit, issue tokens. Repeat use returns `invitation_used`; expired or revoked returns `invitation_invalid`.
6. `GET /admin/invitations` lists with status; expiry is computed on read and finalised by a nightly job.

10. F5 Cohorts, dates and holidays

1. `POST /coordinator/cohorts`: the coordinator creates a cohort for their own department only. Validate `start_date < end_date`, at least one working day, valid settings ranges (grace days 1 to 14, minutes 1 to 3).
2. Save holidays in `cohort_holidays`. Compute week numbers and the total weeks.
3. Attach existing students with the same programme and department who have no cohort; set their `cohort_id` and `coordinator_id`.
4. `PATCH /coordinator/cohorts/{id}`: if any entry or weekly row exists, reject start-date changes unless the actor is admin (`cohort_started`); the end date may only move later; adding a holiday that already has a saved entry is rejected (`holiday_has_entry`).
5. `GET /cohorts/current` returns cohort, week number, day number and `server_time` for the student home.
6. Admin: `GET /admin/cohorts`, `PATCH /admin/cohorts/{id}` (may change both dates; audited).
7. If a department has more than one coordinator, students are assigned to the coordinator who created the matching cohort; admin can reassign (F12).

11. F6 Student home, week view and daily pages

Home and week reads

1. `GET /logbook/home`: resolves the student's cohort, computes today and the current week on the server, **creates missing `weekly_submissions` rows lazily** (idempotent, from week 1 to the current week), and returns: `server_time`, week list with derived `state` (IN_PROGRESS, SUBMITTED, REVIEWED, RETURNED, OVERDUE, MISSED), the current week's day tiles, the deadline timestamp, filled count, `can_submit`, `missing_days`, the supervisor review panel (SIWES: submitted weeks not yet sent, plus past batches), attendance summary (SWEP).
2. Each **day tile** is derived, not stored: `NEW_PAGE` (today, no row), `NOT_FILLED` (past working day this week with no row or empty activities), `DRAFT` (row exists; `ready: true` when activities meets `min_filled_chars`), `SUBMITTED`, `REVIEWED`, `RETURNED`, `FUTURE` (opens later), `HOLIDAY`. No tile is ever labelled missed.
3. `GET /logbook/weeks/{no}` returns one week's tiles and its status; `GET /logbook/days/{date}` returns the page (or an empty blank-page payload without writing a row) plus `editable: true/false` and the reason.

Save a page (autosave and Save draft) `PUT /logbook/days/{date}`

1. The date must be a working day of the cohort, on or before today (server date). Otherwise `future_day` or `not_working_day`.
2. Load the week row `FOR UPDATE`. Require editable (section 3): week OPEN or RETURNED and `now <= effective_deadline`. Otherwise `week_locked`. Earlier weeks are locked unless an approved extension is in force.
3. Sanitise and length-limit `activities`, `skills`, `review_text`. Upsert the row as DRAFT (the first save creates it). Return `updated_at`, `ready` flag and the week's `can_submit`.
4. Concurrent devices: last write wins per request; the response carries `updated_at` and the client sends `If-Unmodified-Since` optionally to detect conflicts (`edit_conflict`).

Photos `POST /logbook/days/{date}/images`, `DELETE /logbook/images/{id}`

1. Same date and lock checks as saving; create the entry row first if absent.
2. Check size (5 MB), MIME by file signature (jpeg, png, webp) and at most 5 per entry. Store under a random key; create a thumbnail in a Celery task.
3. Return a signed URL valid for one hour. Delete removes the row and queues storage deletion under the same lock rules.

12. F7 Weekly submission and deadline

`POST /logbook/weeks/{no}/submit`

1. Lock the week row. If already SUBMITTED, return it unchanged.
2. If `now > effective_deadline`, reject with `deadline_passed` and details: `can_request_extension`, `request_until`.
3. Compute the working days of the week. Every one of them must be **filled**. Otherwise reject with `days_not_filled` and `details.missing_days` (dates). This is the only completeness rule; there is no "missed day".
4. Set the week SUBMITTED, set every entry of the week SUBMITTED, set `submitted_at`; set `is_late = true` when an approved extension applies; clear `correction_deadline` when resubmitting a returned week.
5. Create a notification and email for the coordinator; cancel the week's pending reminders.
6. For a SIWES student the week now counts as waiting to be sent; it is included the next time the student taps Send to supervisor (computed on read).

At the deadline: nothing is written. Any save, upload or submit on a week past `effective_deadline` fails with `week_locked`. Reads return `OVERDUE` while an extension is possible and `MISSED` otherwise. The nightly job finalises approved extensions that ran out as `EXPIRED`.

13. F8 Extension requests

Request `POST /logbook/weeks/{no}/extension-requests`

1. The week must be OVERDUE (OPEN, past `effective_deadline`) and not SUBMITTED.
2. No PENDING request exists for the week (partial unique index also protects against races), the request window is open (`now <= deadline_at + request_window_days`, else `request_window_closed`), and the count of approved extensions is below `max_extensions_per_week` (else `extension_limit`).
3. Validate the reason (at least 20 characters). Attachment goes through the upload flow (pdf, jpeg, png; 5 MB; signature check).
4. Create the request PENDING; notify the coordinator by email and in-app.

Decide `POST /coordinator/extension-requests/{id}/approve` and `/deny`

1. Only the coordinator of that student's department (or admin). Lock the row; only PENDING can change.

2. **Approve** with `grace_days` between 1 and `max_grace_days`: `new_deadline = 23:59:59 Lagos on (today + grace_days)`; set the week's `extended_until`; the week becomes editable again. A week that is part of a supervisor review batch is already submitted, so it cannot have an extension request.
3. **Deny**: comment required; the week stays locked and now derives MISSED.
4. Audit; notify the student by email and in-app.
5. `GET /coordinator/extension-requests` lists requests with student matric number, week, reason, attachment URL (signed) and hours since deadline.

14. F9 SWEP attendance

SIWES users receive `403 not_applicable` on every attendance endpoint.

Generate code `POST /coordinator/attendance/sessions`

1. Coordinator of a SWEP cohort; today must be a working day of the cohort. Body: `minutes` (1 to 3, default 3, cap = `attendance_max_minutes`).
2. Reject with `attendance_limit` if the day already has `attendance_max_codes_per_day` sessions. If a session is still ACTIVE, end it (`ended_at = now`) first; the new code replaces it. Students already PRESENT stay present.
3. Create the code from a 31-character alphabet without look-alikes (six characters). Store `code_hash = SHA-256(salt + code)` and `salt`, `started_at = now`, `expires_at = now + minutes`. Put the plain code in Redis at `att:code:{session_id}` with TTL until `expires_at`, so the coordinator's screen can re-display it. Return `code`, `expires_at`, `server_time`. **The countdown exists only on the coordinator screen.**
4. `GET /coordinator/attendance/sessions/current` returns the code (while active), seconds left and the live present count (the screen polls every 5 seconds). `POST .../{id}/end` ends the session early.

Student enters code `POST /attendance/submit`

1. Student must be SWEP with a cohort; no student timer or attempt-start call exists.
2. Find the cohort's ACTIVE session (`ended_at` null and `now < expires_at`). If none: `attendance_closed` ("Attendance isn't open right now").
3. Count wrong attempts in Redis (`att:tries`); more than 5 per session returns `too_many_attempts`.
4. Compare `SHA-256(salt + submitted)` to `code_hash` in constant time. A wrong or just-expired code returns the generic `code_invalid_or_expired`.

5. On success create `attendance_records` PRESENT (unique per student and date; a repeat returns the existing record), storing session id, time and IP.
6. Return the new status and the student's attendance summary.

Other

- `GET /attendance/history` (student): dates and statuses with the summary (present of days held).
- **Override** `POST /coordinator/attendance/{student_id}/override`: PRESENT or EXCUSED with a required reason; audited.
- **Finalisation** (nightly job): for each SWEP cohort day that had at least one session, students without a record get ABSENT. Days with no session create no records.

15. F10 Coordinator workspace, review and grading

1. `GET /coordinator/dashboard`: counts of weeks to review, extension requests pending, overdue weeks, supervisor details received, and today's attendance session summary.
2. `GET /coordinator/students?filter=to_review|overdue|extension|all&q=`: only students whose `coordinator_id` or department-cohort matches the coordinator; each card returns matric number, weeks submitted of total, badges and counts.
3. `GET /coordinator/students/{id}`: weeks with statuses, missed weeks, running score, SIWES supervisor review batches.
4. `GET /coordinator/weeks/{id}`: only for status SUBMITTED, RETURNED or REVIEWED; returns pages ordered by date with signed image URLs, the student's own review, existing ratings, and the `is_late` flag.
5. `PUT /coordinator/entries/{id}/review`: upsert rating 1 to 5 and comment; allowed while the week is SUBMITTED or RETURNED and not yet finished.
6. `POST /coordinator/weeks/{id}/finish`: requires every written page rated; sets REVIEWED, `reviewed_at`; notifies the student.
7. `POST /coordinator/weeks/{id}/return`: comment required; sets RETURNED, `returned_at`, `correction_deadline = now + correction_days`; entries become RETURNED; notifies the student.
8. **Score**: `average_rating / 5 * 100` over rated entries; also `weeks_reviewed`, `weeks_missed`. A cohort setting `missed_weeks_count_as_zero` (default false) controls whether missed weeks are included as zero. Results are cached on the student's cohort record, recalculated when ratings change and finalised when the cohort closes.
9. Any attempt on a student of another department returns `404`.

A week that belongs to a supervisor review batch with status SENT or REVIEWED cannot be returned (`week_in_supervisor_review`). An admin can void the batch first, which releases its weeks.

16. F11 SIWES supervisor review (send anytime)

Student: send `POST /logbook/supervisor-reviews`

1. Student must be SIWES (`not_applicable` otherwise).
2. **Gather the batch**: every week of the student with status SUBMITTED or REVIEWED that is not yet linked to any supervisor review (no row in supervisor_review_weeks), in week order. Weeks that are OPEN, RETURNED, OVERDUE or MISSED are never included. If the set is empty, return nothing_to_send and create or email nothing. Any programme length and any sending rhythm works.
3. Validate supervisor name, email format and phone (normalised to international format); organisation optional.
4. In one transaction: upsert the student's `supervisors` row; create a supervisor_reviews row (next batch_no) with a snapshot of the supervisor details and link the gathered weeks through supervisor_review_weeks; create a `supervisor_tokens` row (`secrets.token_urlsafe(32)`, hash stored, expires in 14 days); set status SENT and `sent_at`; audit.
5. Queue two emails: to the supervisor (**Leave review** button with the raw token) and to the coordinator of the student's department (supervisor name, email, phone, weeks covered, student). Create the coordinator's in-app notification.
6. The weeks in the batch are already submitted and therefore read-only. The coordinator cannot return a week that belongs to a batch with status SENT or REVIEWED (week_in_supervisor_review); an admin can void the batch first to release its weeks.
7. `POST /logbook/supervisor-reviews/{id}/resend`: at most once every 24 hours while SENT or EXPIRED; invalidates the old token and creates a new one.
8. Raw tokens and hashes never appear in student or coordinator responses.

Supervisor: open `GET /supervisor/review` (header `X-Supervisor-Token`)

1. `SupervisorTokenAuthentication` hashes the token and finds a row. Missing, revoked, used: `token_invalid`; past `expires_at`: `token_expired`; review already submitted: `already_submitted`.
2. Update `last_opened_at` and `open_count`. Throttle by IP and token hash (30 opens per hour).

3. Return a read-only payload for that review batch only (the weeks linked to it): student name, matric number, programme, organisation, the weeks with entries, and signed image URLs valid for one hour. Nothing else; the authentication class cannot reach any other endpoint.
4. Optional cohort setting: require a 6-digit code emailed to the supervisor before the payload loads.

Supervisor: submit `POST /supervisor/review/submit`

1. Validate the token and lock the supervisor review row.
2. Require `review_text` (20 to 4,000 characters) and a signature: a drawn PNG (up to 200 KB, stored privately) or typed name with the confirmation flag.
3. Store review, signature, `signed_name`, `reviewed_at`, IP and user agent. Set status REVIEWED and `used_at` on the token.
4. Notify the student and coordinator; email a receipt to the supervisor. The review is immutable; a coordinator or admin can void it (audited), which releases its weeks so they are included in the next send.
5. `POST /supervisor/review/request-new-link` creates a notification asking the student to resend and does not issue a token.

17. F12 Admin management

1. `GET /admin/overview` counts; `GET /admin/users?role=&q=&status=` with filters and pagination.
2. **Remove student** `PATCH /admin/users/{id}/status`: set WITHDRAWN or RUSTICATED (or DEACTIVATED) with reason and a required note; blacklist all refresh tokens; audit. Records are kept; sign-in is blocked. `DELETE /admin/users/{id}` is a separate, double-confirmed hard delete that cascades and writes an audit entry first.
3. **Change coordinator** `POST /admin/students/{id}/reassign`: sets `coordinator_id`; audited.
4. **Remove coordinator**: reject with `students_assigned` while students remain. `POST /admin/coordinators/{id}/reassign` moves all students, open extension requests and pending reviews to a named coordinator in one transaction; then deactivate and blacklist tokens.
5. **Programme change** `PATCH /admin/students/{id}/programme`: admin only; allowed only while the student has no entries (otherwise `programme_locked`).
6. **Reports** `GET /admin/reports`: submission rate by department, missed weeks, extension counts, attendance averages, logbook score averages, pending reviews; cached for a few minutes. `GET /admin/reports/export.csv` streams CSV.

7. `GET /admin/audit-logs` filters by actor, action, date.
8. Admin can read but not edit student page content.

18. F13 Notifications

- Notifications are rows in `notifications` plus an email task, created by the service that causes the event. `GET /notifications` (paged), `POST /notifications/{id}/read`, `POST /notifications/read-all`.
- Event table: invitation sent (email to coordinator); Friday and Saturday reminders (student); week locked (student); week submitted (coordinator); extension requested (coordinator); extension decided (student); week returned (student); week reviewed (student); supervisor details received (coordinator); review request (supervisor email); supervisor review submitted (student and coordinator); password reset (email).
- Emails are sent by Celery with retry and a dead-letter log; a failed email never rolls back the action that caused it.

19. Scheduled jobs (Celery Beat, Lagos time)

Job	When	What it does
friday_reminder	Fri 16:00	Notify students whose current week is OPEN and not all days are filled or not submitted
saturday_reminder	Sat 18:00	Same, with hours left
extension_followup	Daily 09:00	Remind coordinators of requests pending over 2 days
finalize_extensions	Daily 00:10	Set expired approved extensions to EXPIRED
finalize_attendance	Daily 00:20	Write ABSENT rows for SWEP days that had a session
expire_tokens	Daily 00:30	Expire invitations and supervisor links; mark reviews EXPIRED; notify the student
supervisor_reminder	Daily 09:00	Remind the supervisor on days 5 and 10 of an open request
cohort_close	Daily 00:40	Close cohorts after `end_date`; compute final scores

Jobs are idempotent. Deadline enforcement never depends on them.

20. API summary (prefix /api/v1)

Area	Endpoints
Auth	POST auth/student/signup, auth/login, auth/admin/login, auth/refresh, auth/logout, auth/forgot-password, auth/reset-password; GET me; PATCH me/password
Invitations	POST admin/invitations; GET admin/invitations; POST admin/invitations/{id}/resend; DELETE admin/invitations/{id}; GET invitations/validate; POST invitations/accept
Cohorts	POST coordinator/cohorts; PATCH coordinator/cohorts/{id}; GET cohorts/current; GET admin/cohorts; PATCH admin/cohorts/{id}
Logbook	GET logbook/home; GET logbook/weeks/{no}; GET logbook/days/{date}; PUT logbook/days/{date}; POST logbook/days/{date}/images; DELETE logbook/images/{id}; POST logbook/weeks/{no}/submit
Extensions	POST logbook/weeks/{no}/extension-requests; GET coordinator/extension-requests; POST coordinator/extension-requests/{id}/approve and /deny
Attendance	POST attendance/submit; GET attendance/history; POST coordinator/attendance/sessions; GET coordinator/attendance/sessions/current; POST coordinator/attendance/sessions/{id}/end; POST coordinator/attendance/{student_id}/override
Review	GET coordinator/dashboard; GET coordinator/students; GET coordinator/students/{id}; GET coordinator/weeks/{id}; PUT coordinator/entries/{id}/review; POST coordinator/weeks/{id}/finish and /return

Area	Endpoints
Supervisor review	POST logbook/supervisor-reviews (send) and logbook/supervisor-reviews/{id}/resend; GET logbook/supervisor-reviews (unsent weeks and history); GET supervisor/review; POST supervisor/review/submit; POST supervisor/review/request-new-link
Admin	GET admin/overview; GET admin/users; PATCH admin/users/{id}/status; DELETE admin/users/{id}; POST admin/students/{id}/reassign; PATCH admin/students/{id}/programme; POST admin/coordinators/{id}/reassign; GET admin/reports; GET admin/reports/export.csv; GET admin/audit-logs
Notifications	GET notifications; POST notifications/{id}/read; POST notifications/read-all

21. Permission matrix

Action	Student	Coordinator	Admin	Supervisor
Sign up	Yes	No	No	No
Write pages, submit week	Own	No	No	No
Request extension	Own	No	No	No
Decide extension	No	Own students	Yes	No
Generate attendance code	No	SWEP cohort of own department	No	No
Enter attendance code	SWEP only	No	No	No
Rate, return, finish weeks	No	Own students	Read only	No
Send to supervisor	SIWES only, own	No	No	No
Read one review batch and submit its review	No	No	No	Token only

Action	Student	Coordinator	Admin	Supervisor
Invite coordinators, manage users, cohorts	No	Own cohort only	Yes	No

22. Security requirements

- Tokens for invitations, password reset and supervisor links: 32 random bytes, only the hash stored, expiry enforced, single use, throttled; never logged in full.
- Supervisor access is scoped to one review batch; ids from other batches or students are rejected.
- Attendance codes: hashed with a salt, never returned to student endpoints, wrong-attempt limits, 3-minute maximum life.
- Photos, attachments and signatures are private; signed URLs of one hour.
- Uploads validated by file signature, size and count; random names; no execution.
- ORM parameterisation only; sanitised text; CORS limited to the frontend origin (django-cors-headers); secure cookies; HTTPS only; Django security middleware.
- Personal data of supervisors is used only for the review flow and visible only to the student, their coordinator and admin.
- Daily database backups; error monitoring; structured logs without secrets.

23. Error codes

Code	Meaning
invalid_credentials, account_withdrawn, account_rusticated, account_deactivated	Login failures
matric_taken, email_taken	Signup conflicts
invitation_invalid, invitation_used	Invitation link problems
cohort_started, holiday_has_entry	Cohort edit blocked
future_day, not_working_day	Page not allowed on that date
week_locked	Week submitted, reviewed or past its deadline
days_not_filled	Submit blocked; details list the missing days

Code	Meaning
deadline_passed	Submit after the effective deadline
edit_conflict	Page changed on another device
request_window_closed, extension_limit, week_in_supervisor_review	Extension rules; return blocked for weeks already in a supervisor review
attendance_closed, code_invalid_or_expired, too_many_attempts, attendance_limit	Attendance
nothing_to_send	Send to supervisor tapped but no submitted week is waiting
not_applicable	Action not valid for this programme
token_invalid, token_expired, already_submitted	Supervisor link problems
students_assigned, programme_locked	Admin rules

24. Testing requirements

- Frozen-clock tests around Saturday 23:59:59 Lagos and the UTC conversion; weeks that start mid-week; holidays.
- Submit rules: all days filled, one day short, submit after the deadline, resubmit after return.
- Race tests: two submits, two extension requests, approve and deny together, two attendance submissions.
- Attendance: code valid at 2:59, rejected at 3:01, replaced code, ended code, wrong-attempt limit, SIWES rejected.
- Permission tests per role on every endpoint, including cross-student and cross-department.
- Supervisor token tests: expired, reused, revoked, wrong batch, another student's ids.
- Load test for the Saturday evening submission peak.
- Migration and seed data for cohorts, weeks and review batches.

25. Open technical items

- How missed weeks count in the final percentage (default: excluded, with a setting).
- Whether the supervisor email verification code is required.
- Retention period for photos, signatures and supervisor details after a cohort closes.

- SMS or WhatsApp delivery for supervisor links.
- PDF logbook export with signatures (server-side rendering through Celery).
- Whether the supervisor review carries a numeric score.
- Appeal of a denied extension.